

Atividade 002

Grupo

Leonardo Fratoni Borges da Silva — RA: 24047610-2

Eduardo Viana Marques — RA: 24023663-2

Matheus Costa e Silva — RA: 24000729-2

Pedro Lucas Gomes Braido — RA: 24190443-2

1. Genealogia das linguagens

As linguagens de programação não evoluem simplesmente de uma versão “inferior” para outra “superior”. Muitas linguagens antigas continuam existindo, sendo utilizadas ou influenciando novas linguagens, mesmo depois do surgimento de alternativas mais modernas.

Dois fatores históricos que fazem uma linguagem influenciar outra sem necessariamente substituí-la são:

- **Legado e compatibilidade:** uma linguagem pode permanecer em uso porque existem muitos programas, bibliotecas e sistemas já desenvolvidos nela. Isso torna sua substituição cara e arriscada.
- **Influência de conceitos:** uma linguagem pode introduzir ideias importantes que são incorporadas por outras linguagens, enquanto ela própria continua sendo utilizada. Assim, a nova linguagem aproveita conceitos da anterior sem eliminá-la.

2. Plankalkül

Desenvolvido por Konrad Zuse na década de 1940, é relevante porque foi uma das primeiras propostas de uma linguagem de programação de alto nível, mesmo não tendo sido implementada na época. Seu projeto antecipou várias ideias que posteriormente se tornaram comuns nas linguagens de programação.

Três recursos antecipados pelo projeto foram:

1. **Tipos de dados**, permitindo representar diferentes tipos de valores.
2. **Estruturas de dados**, incluindo a possibilidade de trabalhar com estruturas compostas.
3. **Estruturas de controle**, como condicionais e outras formas de organizar a execução dos

programas.

Um dos recursos mais importantes foi o uso de **tipos e estruturas de dados**, pois isso permitia ao programador representar informações de maneira organizada e abstrata, aproximando a programação da forma como os problemas eram pensados, em vez de exigir apenas operações diretamente relacionadas ao hardware.

3. Short Code, Speedcoding e A-0/A-1/A-2

Os três sistemas buscavam **facilitar a programação**, reduzindo a necessidade de escrever diretamente em código de máquina.

- **Short Code:** utilizava códigos simbólicos para representar operações.
- **Speedcoding:** facilitava a programação, principalmente em cálculos matemáticos.
- **A-0/A-1/A-2:** utilizava sub-rotinas e mecanismos de tradução para gerar código de máquina.

Eles não podem ser chamados simplesmente de compiladores modernos porque eram sistemas pioneiros, específicos e ainda não possuíam todas as características dos compiladores atuais.

Questão 11 — Cadeia ALGOL → Pascal → C versus Prolog

Parte 1: Cadeia de influência imperativa

A ideia dessa cadeia é que uma linguagem aproveita a base da anterior, mas ajusta o foco.

- **ALGOL 60:** consolidou base de programação estruturada (blocos/escopo/recursão) e a BNF.
 - **Pascal:** simplificou o “espírito” do ALGOL e reforçou o uso didático e a tipagem mais forte.
 - **C:** levou essa estrutura para programação de sistemas, com mais controle de memória e menos “segurança”.
-

Parte 2: Contraste com Prolog — paradigma declarativo/lógico

ALGOL/Pascal/C são imperativas: você escreve *como* o programa executa (passo a passo, com atribuição e laços).

Prolog é lógico/declarativo: você descreve *o que é verdade* (fatos e regras) e o sistema busca respostas (backtracking).

- **Controle:** no C você decide a ordem; no Prolog a ordem vem do motor de inferência.
- **Estado:** C usa variáveis mutáveis; Prolog trabalha com variáveis lógicas por unificação.

- **Falha:** em Prolog falhar faz parte da busca; em C normalmente é erro/tratamento de exceção.
-

Questão 12 — Modelo em linguagem natural de base Prolog

A base de conhecimento — fatos e regra

- **Fato:** Maria é professora.
- **Fato:** João é estudante.
- **Regra:** se X é professora e Y é estudante, então X pode ensinar Y.
- **Consulta:** Quem pode ensinar João?

Em Prolog, seria:

```
professora(maria).  
estudante(joao).  
  
pode_ensinar(X, Y) :- professora(X), estudante(Y).  
  
?- pode_ensinar(Quem, joao).  
% Resposta: Quem = maria.
```

Como o Prolog executa essa consulta — passo a passo:

1. Recebe a consulta `pode_ensinar(Quem, joao)` e encontra a regra compatível.
 2. Unifica as variáveis e tenta provar o corpo: `professora(Quem)` e `estudante(joao)`.
 3. Como existe `professora(maria)`, a resposta sai como **Quem = maria**.
-

Por que isso é programação lógica e não apenas banco de dados?

Em banco de dados, você precisa armazenar as relações prontas. No Prolog, a relação é *inferida* a partir de fatos + regras.

- **Inferência:** o sistema deduz resultados que não foram escritos diretamente.
 - **Variáveis lógicas:** a consulta usa incógnitas que o motor resolve por unificação.
 - **Backtracking:** ele tenta alternativas até achar todas as soluções possíveis.
-

4. Fortran: convencer pela economia de tempo, dinheiro e recursos

Eles convenceram mostrando ganho prático: fazer mais rápido e gastar menos máquina/pessoas.

- **Desempenho:** Fortran comprovou que o sistema conseguia otimizar o tempo de código elaborando um sistema baseado em loops, cujo qual simulava os caminhos e organizava os laços de forma quase perfeita, fazendo com que o código rodasse virtualmente na mesma velocidade ou até mais rápido do que um feito manualmente.
 - **Custo/tempo:** Na época a locação de um computador era muito custoso para a empresa, e boa parte desse tempo alugado era utilizado apenas para debugging, o método de Fortran fazia uma redução de linhas de código muito grande, agilizando o processo de desenvolvimento e reduzindo os erros, por conta de ter menos linhas.
 - **Adoção:** A equipe de Fortran enviou o novo compilador para uma fabrica fazer uso, acarretando em Engenheiros não especializados em computação que receberam o material, conseguissem aprender e colocar um programa científico para rodar.
-

5. Fortran e Lisp: domínio, representação e estilo

- **Domínio:** Fortran fez o sistema voltado para projetos envolvendo Física, engenharia e meteorologia.
 - Já Lisp projetou para o quesito de Inteligência Artificial nascente, com objetivo em computar conceitos, lógica, relações linguísticas, entre outros relacionados, onde o retorno não seria somente números, mas símbolos e conexões;
 - **Representação:** para Fortran os dados possuem tamanhos fixos, permitindo que o computador saiba onde localizar a informação do valor instantaneamente.
 - já para Lisp, o tamanho dos valores são variáveis, podendo aumentar ou diminuir, além de poder guardar qualquer informação, seja ela palavras, números e outras listas.
 - **Estilo:** Fortran usava um estilo onde daria ordens sequenciais explícitas através de loops, fazendo com que o programador gerencie o estado de memória, com foco em como a máquina deve executar de forma linear.
 - Já Lisp usava um estilo onde ele implementa um função que chama a si mesma, ao invés de utilizar laços de repetição, assim favorecendo imutabilidade e a transformação de dados.
-

14. Compare o papel dos objetos em Smalltalk, C++ e Java. Inclua na resposta o compromisso de C++ com C e a estratégia de portabilidade de Java.

- **Smalltalk:** tudo é tratado como **objeto**, inclusive números e estruturas básicas. A comunicação acontece principalmente por **envio de mensagens** entre objetos.
- **C++:** utiliza objetos e classes, mas mantém forte compatibilidade com **C**. Por isso, permite programação orientada a objetos sem abandonar recursos de programação procedural e de baixo nível, como ponteiros e acesso direto à memória.
- **Java:** é fortemente orientada a objetos e busca **portabilidade**. O código é compilado para **bytecode**, executado pela **JVM**, permitindo que o mesmo programa rode em diferentes sistemas operacionais sem precisar ser recompilado para cada um.

18. Diferencie XSLT e JSP quanto a entrada, processamento e saída. Por que ambas podem ser chamadas de linguagens híbridas de marcação e programação?

- **XSLT**: recebe como **entrada um documento XML**. Durante o processamento, aplica regras de transformação definidas no XSLT e gera uma **saída**, que pode ser HTML, XML ou texto.
- **JSP**: recebe como **entrada uma requisição HTTP**, geralmente associada a dados de uma aplicação. Durante o processamento, mistura HTML com código Java e gera como **saída uma página HTML** enviada ao navegador.

Por que são linguagens híbridas?

As duas combinam **marcação e programação**. O XSLT utiliza elementos de marcação XML junto com instruções de programação, como condições e repetições, para transformar dados. O JSP mistura marcação HTML com código Java para gerar páginas dinamicamente.

19. Linha do tempo com oito linguagens e tipos de influência

- **1957 – Fortran**: uma das primeiras linguagens de alto nível, influenciando principalmente linguagens **imperativas** e científicas.
- **1958 – Lisp**: introduziu conceitos importantes da **programação funcional** e influenciou diversas linguagens funcionais.
- **1958 – ALGOL**: influenciou a estrutura de várias linguagens **imperativas**, incluindo Pascal e C.
- **1967 – Simula**: introduziu conceitos fundamentais da **orientação a objetos**, influenciando Smalltalk e C++.
- **1972 – Smalltalk**: desenvolveu a orientação a objetos de maneira mais completa, influenciando posteriormente linguagens como Java.
- **1972 – C**: tornou-se muito importante na programação de sistemas e influenciou diretamente o desenvolvimento de C++.
- **1985 – C++**: acrescentou orientação a objetos ao C e influenciou linguagens posteriores, especialmente Java.
- **1995 – Java**: combinou **orientação a objetos** com uma estratégia de **portabilidade**, utilizando a JVM.
-

Influências principais

- **ALGOL** → influenciou **C**;
- **Simula** → influenciou **Smalltalk** e **C++**;
- **C** → influenciou **C++**;
- **Smalltalk** e **C++** → influenciaram **Java**;
- **Lisp** → influenciou a **programação funcional**.

